PLANNING D'IMPLÉMENTATION — Assistant Matanne

Basé sur : ANALYSE_COMPLETE.md (audit du 29 mars 2026)
Objectif : Transformer l'audit exhaustif en plan d'implémentation structuré, actionnable et priorisé
Portée : Backend (FastAPI/Python), Frontend (Next.js/TypeScript), SQL, Tests, Documentation, IA, Automatisations, Nouveau module Habitat

Table des matières

1. État des lieux & Métriques de référence
2. Architecture actuelle par module
3. Consolidation SQL — Actions requises
4. Corrections de bugs — Plan de résolution
5. Comblement des gaps fonctionnels
6. Interactions intra-module — Référence existante
7. Interactions inter-modules — Référence existante
8. Interactions inter-modules — Implémentations à réaliser
9. Plan d'enrichissement IA
10. Jobs automatiques & Cron — Plan d'action
11. Notifications — Plan de consolidation
12. Mode Admin — Plan d'implémentation
13. Moteur d'automatisation — Plan d'extension
14. Tests — Plan d'amélioration
15. Documentation — Plan de mise à jour
16. Organisation du code & Résorption de la dette technique
17. Axes d'amélioration & Innovation
18. Plan d'action global priorisé par phase
19. Nouveau module — Projet Habitat
20. Annexe A — Tableau de synthèse des fichiers clés
21. Annexe B — Checklist review avant implémentation

1. État des lieux & Métriques de référence

Inventaire complet (point de départ)

Dimension	Quantité	État	Objectif cible
Pages frontend	57 pages (+ 2 auth)	✅ Toutes implémentées	Maintenir
Routes API	300+ endpoints (36 fichiers routeurs)	✅ Complet	Maintenir + Habitat
Schémas Pydantic	150+ classes (23 fichiers)	⚠️ 8 classes stub (`pass`)	0 stubs
Modèles SQLAlchemy	80+ modèles (30 fichiers)	✅ Complet	+ modèles Habitat
Tables SQL	150+ tables	✅ Complet	+ tables Habitat
Services métier	100+ services (factory pattern)	✅ Complet	+ services Habitat & IA
Fichiers de tests	186 fichiers	⚠️ Gaps sur admin/cron/intégrations	Couverture ≥ 80%
Cron jobs	19 tâches planifiées	✅ Fonctionnel	+ 8 jobs manquants
Documentation	25 fichiers markdown	⚠️ Guides modules datés	Tout à jour
Composants UI	29 shadcn/ui + 40 spécialisés	✅ Complet	WCAG conforme
Hooks React	12 hooks custom	✅ Complet	Maintenir
Stores Zustand	4 stores + 3 tests	✅ Minimal mais suffisant	Maintenir
Clients API frontend	23 fichiers	⚠️ Manque `admin.ts`	+ admin.ts + habitat.ts

Scores actuels → Objectifs cibles

Dimension	Score actuel	Objectif	Actions principales
Architecture	9/10	9/10	Maintenir
Sécurité	9/10	9.5/10	Guards admin frontend
Typage	8/10	9/10	Éliminer `# type: ignore` , compléter stubs
Tests	6/10	8/10	+150 tests ciblés (admin, cron, WhatsApp, multi-user)
Documentation	6/10	8/10	8 docs manquantes + mise à jour guides
Couverture IA	7/10	9/10	10 services à enrichir + 7 nouvelles fonctionnalités
Inter-modules	5/10	8/10	8 interactions manquantes à implémenter
Notifications	6/10	8/10	Failover, préférences unifiées, throttling
Automatisations	4/10	7/10	10 triggers, 9 actions, historique, dry-run

2. Architecture actuelle par module

2.1 Module Cuisine (12 services, 11 pages, 25+ endpoints)

Couche	Fichiers	État
Routes	`recettes.py` , `courses.py` , `planning.py` , `inventaire.py` , `batch_cooking.py` , `suggestions.py` , `anti_gaspillage.py`	✅
Services	recettes, courses, planning, suggestions, batch_cooking, nutrition_enrichment, import_recettes, recurrence, rappels, photo_frigo, prediction_peremption, prediction_courses	✅
Modèles	Recette, Ingredient, Etape, ListeCourses, ArticleCourses, Planning, Repas, ArticleInventaire, SessionBatchCooking	✅

Couche	Fichiers	État
Pages frontend	Hub + recettes (list/detail/create) + courses + planning + inventaire + batch-cooking + anti-gaspillage + nutrition + scan-ticket	✅
Tests	`tests/services/recettes/` (5), `tests/services/planning/` (8), `tests/services/batch_cooking/` (3), `tests/services/inventaire/` (2)	⚠️ Manque anti-gaspillage

Actions Cuisine :

- ☐ Tests anti-gaspillage à ajouter
- ☐ G-CUI-1 : Péremption → suggestions recettes auto
- ☐ G-CUI-2 : Batch cooking → courses auto
- ☐ G-CUI-3 : Nutrition tracking hebdomadaire
- ☐ IA-1 : Enrichir NutritionEnrichmentService avec détection carences

2.2 Module Famille (23 services, 18 pages, 18+ endpoints)

Couche	Fichiers	État
Routes	`famille.py` (route monolithique avec sous-routes enfants, activités, jalons, santé, budget, anniversaires)	✅
Services	jules, jules_ai, activites, anniversaires, budget, budget_ai, calendrier, contacts, carnet_sante, documents, evenements, routines, sante, suivi_perso, weekend, weekend_ai, voyage, achats_ia, rappels_famille, resume_hebdo, soiree_ai, version_recette_jules, checklists_anniversaire	✅
Modèles	ProfilEnfant, Jalon, ActiviteFamille, BudgetFamille, AnniversaireFamille, EvenementFamilial, Voyage, etc.	✅
Pages frontend	Hub + jules + activites + routines + budget + weekend + album + anniversaires + contacts + documents + calendrier + gamification + garmin + journal + config + timeline + voyages	✅
Tests	Couverture partielle — manque tests unitaires pour budget_ai, achats_ia, soiree_ai	⚠️

Actions Famille :

- ☐ Tests unitaires pour budget_ai, achats_ia, soiree_ai
- ☐ G-FAM-1 : Routines → Planning (mapper heure_prevue vers créneaux)
- ☐ G-FAM-2 : Budget famille + maison = budget global
- ☐ G-FAM-3 : Anniversaires → suggestions cadeaux IA
- ☐ DT-5 : Splitter `famille.py` en sous-routeurs

2.3 Module Maison (15+ services, 17 pages, 35+ endpoints)

Couche	Fichiers	État
Routes	`maison.py` (route monolithique : projets, entretien, jardin, artisans, contrats, cellier, dévis, diagnostics, énergie)	✅
Services	projets, entretien, jardin, artisans, cellier, visualisation_maison, devis, nuisibles_crud, devis_crud, entretien_saisonnier_crud, releves_crud, conseiller_maison, catalogue_entretien	✅
Modèles	Projet, TacheProjet, Routine, ElementJardin, Meuble, StockMaison, Contrat, Artisan, DiagnosticMaison, etc. (25+ modèles)	✅
Pages frontend	Hub + travaux + jardin + menage + finances + charges + provisions + artisans + equipements + contrats + diagnostics + documents + meubles + visualisation + energie	✅

Actions Maison :

- ☐ G-MAI-1 : Énergie → anomalies IA
- ☐ G-MAI-2 : Jardin → météo proactive (alertes → actions)
- ☐ G-MAI-3 : Exposer les 4 vues SQL non exploitées
- ☐ G-MAI-5 : Maintenance prédictive (patterns saisonniers)
- ☐ DT-4 : Splitter `maison.py` en sous-routeurs

2.4 Module Jeux (12+ services, 6 pages, 40+ endpoints)

Couche	Fichiers	État
Routes	`jeux.py` (paris CRUD, loto, euromillions, predictions, stats, backtest, KPIs)	✅
Services	paris_crud, loto_crud, euromillions_crud, prediction, backtest, jeux_ai, sync, scheduler, loto_data, football_data, responsable_gaming, series	✅

Couche	Fichiers	État
Modèles	Equipe, Match, PariSportif, TirageLoto, GrilleLoto, StatistiquesLoto, SerieJeux, AlerteJeux, ConfigurationJeux	✅
Pages frontend	Hub + paris + loto + euromillions + performance + ocr-ticket	✅

Actions Jeux :

- ☐ G-JEU-1 : Pertes → budget/finances (sync auto)
- ☐ G-JEU-2 : Page frontend bankroll management
- ☐ G-JEU-3 : Connecter alertes responsable gaming aux canaux notifications

2.5 Module Outils (8 pages, services variés)

Couche	Fichiers	État
Routes	`utilitaires.py` (notes, journal, contacts, liens, mdp, énergie), `assistant.py` (chat IA)	✅
Services	notes, journal, contacts, liens, mots_de_passe, energie, chat_ia, ocr, meteo, image_generator	✅
Pages frontend	Hub + chat-ia + assistant-vocal + automations + convertisseur + meteo + minuteur + notes + nutritionniste	✅

Actions Outils :

- ☐ G-OUT-1 : Notes → tags/catégories
- ☐ G-OUT-2 : Chat IA → contexte multi-module
- ☐ G-OUT-3 : Minuteur → intégration recettes

2.6 Dashboard & Admin

Couche	Fichiers	État
Routes	`dashboard.py` , `admin.py`	✅
Services	accueil_data, score_bien_etre, resume_famille_ia, points_famille, anomalies_financieres	✅

Couche	Fichiers	État
Pages frontend	Dashboard principal, Admin (audit/jobs/notifications/services/users)	✅
Documentation	Admin : aucune doc	❌

Actions Dashboard & Admin :

- [ ] G-DASH-1 : Vue budgétaire unifiée (famille + maison + jeux)
- [ ] G-DASH-2 : Score bien-être interactif
- [ ] G-DASH-3 : Widgets configurables
- [ ] DT-6 : Client API admin frontend
- [ ] DOC-1 : Créer ADMIN_RUNBOOK.md

3. Consolidation SQL — Actions requises

3.1 Structure actuelle (référence)

Le fichier `sql/INIT_COMPLET.sql` (~4 500 lignes) contient :

Section	Contenu	Lignes approx.
Part 1	Extensions + schema versioning	1-200
Part 2	Table `schema_migrations`	200-250
Part 3	45+ tables autonomes (sans FK)	250-1500
Part 4	30+ tables avec FK	1500-2500
Part 5A	Tables maison (jardin, tâches)	2500-2900
Part 5B	Extensions jeux (euromillions, bankroll)	2900-3200
Part 5C	Extensions maison (contrats, artisans, garanties)	3200-3600
Part 5D	Utilitaires (notes, journal, contacts, énergie)	3600-3900
Part 6	Triggers (auto-update `modifie_le`)	3900-4100
Part 7	Vues + fonctions helper	4100-4400

Section	Contenu	Lignes approx.
Part 8	Requêtes analytics/reporting	4400-4500

3.2 Inventaire des tables par domaine

Domaine	Tables	Exemples
Cuisine	15	`recettes` , `ingredients` , `etapes` , `listes_courses` , `articles_courses` , `plannings` , `repas_planning` , `inventaire` , `batch_cooking_sessions`
Famille	20	`profils_enfants` , `jalons` , `activites_famille` , `budgets_famille` , `anniversaires_famille` , `evenements_familiaux` , `vaccins` , `documents_famille`
Maison	25	`projets_maison` , `taches_projets` , `zones_jardin` , `plantes_jardin` , `contrats` , `artisans` , `garanties` , `diagnostics_maison` , `meubles`
Jeux	15	`jeux_matchs` , `jeux_paris_sportifs` , `jeux_tirages_loto` , `jeux_grilles_loto` , `jeux_equipes` , `jeux_bankroll` , `jeux_series`
Finances	8	`depenses` , `budgets_mensuels` , `factures` , `depenses_maison`
Planning/Calendrier	8	`evenements_planning` , `calendriers_externes` , `evenements_calendrier` , `voyages`
Gamification	3	`points_utilisateurs` , `badges_utilisateurs` , `jeux_bankroll_historique`
Santé/Garmin	5	`garmin_tokens` , `activites_garmin` , `resumes_quotidiens_garmin` , `journaux_alimentaires`
Notifications	5	`abonnements_push` , `preferences_notifications` , `webhooks_abonnements` , `historique_actions`
Admin/Sécurité	5	`logs_securite` , `etats_persistants` , `automations`
Utilitaires	8	`notes_memos` , `journal_bord` , `contacts_utiles` , `liens_favoris` , `mots_de_passe_maison` , `releves_energie`

Domaine	Tables	Exemples
TOTAL	**~150**	

3.3 Problèmes de cohérence SQL détectés

#	Problème	Sévérité	Tables concernées
S1	**Typage `user_id` incohérent** : UUID dans certaines tables, VARCHAR dans d'autres, INTEGER dans d'autres	🔴 HAUTE	Multi-tables
S2	**Doublon legacy** `liste_courses` vs `listes_courses` — table legacy non nettoyée	🟡 MOYENNE	Courses
S3	**FK CASCADE incohérent** : mix de `ON DELETE SET NULL` et `ON DELETE CASCADE` sans logique claire	🟡 MOYENNE	Divers
S4	**Champs JSONB non typés** : `preferences_utilisateurs`, `preferences_home` stockent des JSON sans validation SQL	🟢 BASSE	Preferences
S5	**4 migrations absorbées** dans INIT_COMPLET mais encore référencées dans `schema_migrations`	🟢 BASSE	Migrations

3.4 Plan de consolidation SQL — Actions concrètes

> **Contexte** : En dev, pas de migration formelle. Tout doit être regroupé dans `INIT_COMPLET.sql`.

#	Action	Effort	Détail
SQL-1	**Standardiser `user_id`** → UUID partout	2h	Type `UUID DEFAULT gen_random_uuid()` dans toutes les tables
SQL-2	**Nettoyer tables legacy**	30min	Supprimer `liste_courses` (ancienne), garder `listes_courses`
SQL-3	**Uniformiser CASCADE**	1h	Convention : `CASCADE` pour enfants forts (ingrédients d'une recette), `SET NULL` pour références faibles (artisan d'un contrat)

#	Action	Effort	Détail
SQL-4	**Ajouter index manquants**	1h	Index composites pour jointures fréquentes (planning ↔ recette, courses ↔ inventaire)
SQL-5	**Documenter les vues SQL**	30min	Ajouter commentaires dans INIT_COMPLET
SQL-6	**Ajouter contraintes CHECK pour enums**	1h	Valider les enums côté SQL (pas juste côté Pydantic)

3.5 Vues SQL existantes — Plan d'exploitation

Vue	Usage	État actuel	Action
`v_objets_a_remplacer`	Objets à remplacer (priorité)	⚠️ Pas dans le frontend	Exposer via endpoint + widget dashboard
`v_temps_par_activite_30j`	Temps par activité (30 jours)	⚠️ Pas dans le frontend	Exposer dans dashboard famille
`v_budget_travaux_par_piece`	Budget travaux par pièce	⚠️ Pas dans le frontend	Exposer dans maison/finances
`v_taches_jour`	Tâches du jour (jointure zones/pièces)	✅ Utilisée dans dashboard	Maintenir
`v_charge_semaine`	Charge de travail hebdomadaire	⚠️ Pas dans le frontend	Exposer dans dashboard planning

4. Corrections de bugs — Plan de résolution

4.1 Bugs critiques P0 — Fix immédiat

#	Bug	Localisation	Fix à appliquer
B1	**Import `ActiviteFamille` cassé** dans `recherche.py` — modèle renommé/déplacé	`src/api/routes/recherche.py`	Corriger le chemin d'import vers le bon modèle
B2	**Cron job référence `liste_courses`** au lieu de `articles_courses`	`src/services/core/cron/jobs.py`	Remplacer la référence par `articles_courses`

4.2 Bugs importants P1 — Sprint correctif

#	Bug	Localisation	Fix à appliquer
B3	**30+ `# type: ignore`** masquent des erreurs	services/jeux/, services/maison/	Remplacer par `cast()` ou corriger les types
B4	**8 schémas Pydantic `pass`** (classes vides)	`src/api/schemas/utilitaires.py`, `documents.py`	Compléter avec les champs corrects
B5	**Hardcodé `items` fallback** dans le parser IA	`src/core/ai/parser.py`	Rendre le fallback configurable
B6	**Hardcodé `USER_ID="matanne"`** dans cron jobs	`src/services/core/cron/jobs.py`	Boucler sur tous les users

4.3 Bugs mineurs P2 — Backlog

#	Bug	Localisation	Fix à appliquer
B7	**Race condition** sur	`src/services/cuisine/partage_recettes.py`	Utiliser un verrou par ressource

#	Bug	Localisation	Fix à appliquer
	`_verrou_tokens` (Lock global)		
B8	**50+ stubs `pass`** — fonctions non implémentées	Divers services	Implémenter ou supprimer
B9	**Erreurs WebSocket silencieuses**	`src/api/websocket_courses.py`	Ajouter retry automatique
B10	**Dashboard alertes avalées** — exceptions → `[]` silencieux	`src/services/dashboard/service.py`	Logger + retourner erreur partielle
B11	**Frontend guards admin manquants**	`frontend/src/app/(app)/admin/`	Ajouter vérification `role === "admin"`
B12	**20+ violations accessibilité** — ARIA incorrects	`frontend/src/composants/ui/`	Corriger attributs ARIA

4.4 Résumé des corrections

Priorité	Nombre	Statut
P0 — Bloquant	2	🔴 À faire immédiatement
P1 — Important	4	🟡 Sprint correctif
P2 — Mineur	6	🟢 Backlog
Total	**12**	

5. Comblement des gaps fonctionnels

5.1 Cuisine — Gaps à combler

Gap	Description	Priorité	Phase
G-CUI-1	**Anti-gaspillage → recettes auto** : items proches péremption → suggestions recettes auto → ajout planning	HAUTE	Phase 5
G-CUI-2	**Batch cooking → courses auto** : session batch cooking → génération auto liste courses	MOYENNE	Phase 5
G-CUI-3	**Nutrition tracking continu** : suivi nutritionnel hebdomadaire (pas juste par recette)	MOYENNE	Phase 6
G-CUI-4	**Import recettes depuis photo** : photo frigo → ingrédients → import recettes en un clic	BASSE	Phase 10
G-CUI-5	**Historique des plannings** : visualisation plannings passés pour détecter patterns	BASSE	Phase 10

5.2 Famille — Gaps à combler

Gap	Description	Priorité	Phase
G-FAM-1	**Routines → Planning** : routines familiales bloquent des créneaux dans le planning. **Diagnostic** : `TacheRoutine.heure_prevue` (HH:MM) jamais croisé avec `Repas.type_repas` (enum). `ServicePlanningUnifie._charger_routines()` regroupe tout sous clé `"routine_quotidienne"` sans distinction. **Fix** : (1) mapper `heure_prevue` vers créneau, (2) vérifier conflits, (3) cron sync quotidien	HAUTE	Phase 5
G-FAM-2	**Budget → Dashboard intégré** : budget famille non agrégé avec budget maison dans un budget global	MOYENNE	Phase 5
G-FAM-3	**Rappels anniversaires → achat cadeau** : pas de lien entre anniversaire et suggestions cadeaux IA	MOYENNE	Phase 6

5.3 Maison — Gaps à combler

Gap	Description	Priorité	Phase
G-MAI-1	**Énergie → anomalies IA** : collecte sans analyse, pas de détection anomalies consommation	HAUTE	Phase 6
G-MAI-2	**Jardin → météo proactive** : alertes météo sans déclenchement d'actions (arrosage, protection)	MOYENNE	Phase 5
G-MAI-3	**Vues SQL non exploitées** : 4 vues SQL créées mais jamais affichées dans le frontend	MOYENNE	Phase 5
G-MAI-5	**Maintenance prédictive** : tâches entretien statiques au lieu d'apprendre patterns saisonniers	BASSE	Phase 6

5.4 Jeux — Gaps à combler

Gap	Description	Priorité	Phase
G-JEU-1	**Pertes → budget** : pertes paris non reflétées dans le module budget/finances	HAUTE	Phase 5
G-JEU-2	**Bankroll management** : concept en DB mais pas de page frontend dédiée	MOYENNE	Phase 9
G-JEU-3	**Alertes responsable gaming** : seuils existent mais notifications non connectées aux canaux	MOYENNE	Phase 8

5.5 Outils — Gaps à combler

Gap	Description	Priorité	Phase
G-OUT-1	**Notes → tags/catégories** : pas de système de tags pour organiser les notes	MOYENNE	Phase 9
G-OUT-2	**Chat IA → contexte multi-module** : chat IA ne connaît pas le contexte utilisateur	HAUTE	Phase 6
G-OUT-3	**Minuteur → intégration recettes** : minuteur non lié aux étapes de recettes	MOYENNE	Phase 10

5.6 Dashboard — Gaps à combler

Gap	Description	Priorité	Phase
G-DASH-1	**Vue budgétaire unifiée** : pas d'agrégation budget famille + maison + jeux	HAUTE	Phase 5
G-DASH-2	**Score bien-être** : calculé côté service mais pas affiché interactivement	MOYENNE	Phase 9
G-DASH-3	**Widgets configurables** : pas de personnalisation disposition widgets	BASSE	Phase 10

6. Interactions intra-module — Référence existante

Interactions **à l'intérieur** d'un même module qui fonctionnent déjà — pas d'action requise, référence pour la maintenance.

Cuisine (intra)

Interaction	Flux	État
Recettes → Planning	Catalogue recettes → planifier la semaine	✅
Planning → Courses	Planning repas → générer liste courses automatiquement	✅
Inventaire → Anti-gaspillage	Items périmés → alertes anti-gaspillage	✅ Partiel
Recettes → Batch cooking	Sélection recettes → session batch cooking	✅
Suggestions IA → Planning	IA suggère des recettes → insertion dans le planning	✅
Photo frigo → Suggestions	Analyse photo → ingrédients détectés → suggestions recettes	✅

Famille (intra)

Interaction	Flux	État
Jules (enfant) → Jalons	Suivi développement → jalons atteints	✅
Jules → Recettes adaptées	Portions enfant ajustées automatiquement	✅
Budget → Achats	Suivi achats → impact sur budget famille	✅
Anniversaires → Checklists	Anniversaire créé → checklist auto-générée	✅
Weekend IA → Activités	Suggestions IA de weekend → activités planifiées	✅

Maison (intra)

Interaction	Flux	État
Projets → Tâches	Projet créé → décomposition en tâches	✅
Entretien → Calendrier	Tâche entretien → planification récurrente	✅
Artisans → Contrats	Artisan sélectionné → contrat/devis lié	✅
Diagnostics → Projets	Diagnostic détecté → projet de réparation suggéré	✅

Jeux (intra)

Interaction	Flux	État
Paris → Stats	Paris placés → statistiques de performance	✅
Predictions IA → Paris	IA prédit résultats → aide à la décision	✅
Backtest → Stratégies	Backtesting historique → validation stratégies	✅
Séries → Alertes	Détection patterns → alertes jeu responsable	✅

7. Interactions inter-modules — Référence existante

Interactions **entre modules différents** qui fonctionnent déjà — référence pour la maintenance.

#	Source → Destination	Mécanisme	État
IM-1	**Planning Cuisine → Courses**	Auto-génération de la liste de courses depuis le planning semaine	✅ Complet
IM-2	**Famille (Jules) → Cuisine**	Portions enfant dans les recettes, versions recettes adaptées	✅ Complet
IM-3	**Maison Entretien → Planning**	Tâches entretien saisonnier intégrées au planning général	✅ Complet
IM-4	**Dashboard ← Tous les modules**	Agrégation en lecture seule depuis cuisine, famille, maison, jeux	✅ Complet
IM-5	**Gamification ← Multi-module**	Points et badges accordés pour actions dans cuisine, famille, maison	✅ Complet
IM-6	**Cron jobs → Notifications**	19 jobs planifiés déclenchent des notifications cross-module	✅ Complet
IM-7	**Planning → Voyage**	Service dédié `inter_module_planning_voyage.py` pour synchroniser	✅ Complet

8. Interactions inter-modules — Implémentations à réaliser

Priorité HAUTE — À implémenter en Phase 5

#	Interaction	Bénéfice	Complexité	Implémentation
IM-8	**Jeux (pertes/gains) → Budget/Finances**	Résultats paris → budget global auto. Alerte si seuil perte atteint	Moyenne	Service sync + cron quotidien
IM-9	**Inventaire péremption →**	Items proches expiration → IA	Faible	Enrichir cron `peremptions_alert`

#	Interaction	Bénéfice	Complexité	Implémentation
	Recettes suggestions	suggère recettes → auto-ajout planning		
IM-10	**Routines famille → Planning général**	Routines récurrentes (école, activités) → créneaux bloqués planning central	Moyenne	Mapper `heure_prevue` → créneau + cron sync
IM-11	**Chat IA contexte → Tous modules**	Chat IA connaît planning, inventaire, budget, météo	Élevée	Enrichir contexte BaseAIService

Priorité MOYENNE — Phase 5-8

#	Interaction	Bénéfice	Complexité	Implémentation
IM-12	**Jardin récoltes → Cuisine inventaire**	Récoltes auto-ajoutées à l'inventaire → suggestions recettes saisonnières	Faible	Événement jardin → inventaire
IM-13	**Énergie → Batch cooking**	Optimiser sessions batch cooking pour heures creuses	Moyenne	Enrichir batch cooking service
IM-14	**Météo → Activités famille**	Météo défavorable → suggestions activités intérieur	Faible	Enrichir cron `push_contextuel_soir`
IM-15	**Push ↔ WhatsApp failover**	Si push échoue → fallback WhatsApp. Préférences canal unifiées	Moyenne	NotifDispatcher avec chaîne fallback

Priorité BASSE — Phase 10

#	Interaction	Bénéfice	Complexité	Implémentation
IM-16	**Garmin santé → Nutrition**	Données activité physique → ajuster suggestions nutritionnelles	Élevée	Enrichir NutritionService
IM-18	**Maintenance prédictive → Budget maison**	Prédire coûts maintenance basés sur historique	Élevée	ML sur données entretien
IM-19	**Predictions courses → Auto-commande**	Prédictions achat → pré-remplir liste courses auto	Moyenne	Enrichir PredictionCourses

9. Plan d'enrichissement IA

9.1 Services avec IA existants — 11 services (référence)

Service	Usage IA	Modèle
`BaseAIService`	Fondation : rate limiting, cache, parsing, streaming	Mistral
`ChatAIService`	Chat conversationnel multi-contexte	Mistral
`BilanMensuelService`	Résumé mensuel IA de la famille	Mistral
`ResumeFamilleIAService`	Résumé hebdomadaire famille	Mistral
`AnomaliesFinancieresService`	Détection anomalies budget	Mistral
`JulesAIService`	Suggestions développement enfant	Mistral
`WeekendAIService`	Suggestions activités weekend	Mistral
`SoireeAIService`	Suggestions soirées couple	Mistral
`JeuxAIService`	Prédictions paris sportifs	Mistral

Service	Usage IA	Modèle
`ConseillierMaisonService`	Conseils entretien maison	Mistral
`ProjetsService`	Estimation projets par IA	Mistral

9.2 Services SANS IA à enrichir — 10 candidats

#	Service	Enrichissement IA proposé	Impact	Phase
IA-1	**NutritionEnrichmentService**	OpenFoodFacts uniquement → Ajouter détection carences + suggestions recettes compensatoires	🔴 HAUTE	Phase 6
IA-2	**ServicePredictionCourses**	Stats pures → Apprendre **pourquoi** les items sont achetés (saison, invités, événements)	🔴 HAUTE	Phase 6
IA-3	**ServicePredictionPeremption**	Règles statiques → Apprendre patterns consommation réels du ménage	🟡 MOYENNE	Phase 6
IA-4	**RappelsIntelligentsService**	Règles hardcodées → Apprendre préférences timing notification	🟡 MOYENNE	Phase 7
IA-5	**ServiceConflits**	Détection rule-based → Générer solutions créatives de résolution	🟡 MOYENNE	Phase 7
IA-6	**OcrService**	Texte brut → Extraire noms de recettes et auto-importer	🟡 MOYENNE	Phase 7
IA-7	**EnergieService**	Collecte uniquement → Prédiction anomalies + alertes	🟡 MOYENNE	Phase 6
IA-8	**MeteoService**	Données météo → Suggestions activités	🟢 BASSE	Phase 10

#	Service	Enrichissement IA proposé	Impact	Phase
		contextuelles		
IA-9	**ChecklistVoyageService**	Checklists statiques → Personnalisées par IA (destination, saison, famille)	🟢 BASSE	Phase 10
IA-10	**NotificationsMaisonService**	Templates alertes → Prédire urgences maintenance proactivement	🟢 BASSE	Phase 10

9.3 Nouvelles fonctionnalités IA à implémenter

#	Fonctionnalité	Module(s)	Description	Phase
IA-NEW-1	**Assistant multi-contexte**	Outils + Tous	Chat IA connaissant planning, inventaire, budget, météo. "Que faire pour le dîner ce soir avec ce qu'il reste dans le frigo et le budget serré ?"	Phase 6
IA-NEW-2	**Recommandation budget**	Finances	IA analyse dépenses sur 3 mois → optimisations catégorie par catégorie	Phase 6
IA-NEW-3	**Planificateur semaine complet**	Planning	IA génère semaine entière : repas + activités + tâches maison + courses (météo + budget)	Phase 6
IA-NEW-4	**Coach bien-être familial**	Famille	Score bien-être IA + recommandations personnalisées (sommeil, activité, nutrition)	Phase 10
IA-NEW-5	**Détection de tendances**	Dashboard	IA détecte patterns sur 3-6 mois : "Vous dépensez 30% de plus en hiver", "Jules progresse plus vite en motricité"	Phase 6
IA-NEW-6	**Génération d'images recettes**	Cuisine	Illustrations recettes sans photo (Mistral Pixtral ou DALL-E)	Phase 10

#	Fonctionnalité	Module(s)	Description	Phase
IA-NEW-7	**OCR intelligent factures**	Maison	OCR + IA pour extraire catégorie, montant, récurrence depuis photo facture	Phase 10

10. Jobs automatiques & Cron — Plan d'action

10.1 Jobs existants (19) — Référence

Job ID	Fréquence	Heure	Fonction	Module
`rappels_famille`	Quotidien	07:00	Rappels anniversaires, documents, école	Famille
`rappels_maison`	Quotidien	08:00	Rappels garanties, contrats, entretien	Maison
`rappels_generaux`	Quotidien	08:30	Alertes inventaire, rappel repas du jour	Cuisine
`peremptions_alert`	Quotidien	09:00	Produits proches péremption	Cuisine
`sync_garmin`	Quotidien	06:00	Sync données santé Garmin	Famille
`push_contextuel_soir`	Quotidien	18:00	Push notifications résumé soirée	Multi

Job ID	Fréquence	Heure	Fonction	Module
`rappel_courses`	Quotidien	18:00	Rappel panier courses (ntfy + WhatsApp)	Cuisine
`rappel_routes`	Quotidien	19:00	Rappels routes/trajets	Planning
`resume_hebdo`	Hebdo	Dim 19:00	Résumé hebdomadaire par email	Multi
`entretien_saisonnier`	Hebdo	Lun 06:00	Vérification entretien saisonnier	Maison
`scores_gamification_reset`	Hebdo	Lun 00:00	Reset points hebdomadaires	Gamification
`archiver_documents`	Hebdo	Ven 22:00	Archivage documents anciens	Famille
`check_alertes_jeux`	Toutes les 6h	:00,:06,:12,:18	Alertes séries/jeu responsable	Jeux
`sync_calendrier_externe`	Toutes les 4h	:15	Sync Google/Outlook	Calendrier
`nettoyage_cache`	Horaire	:00	Purge cache expiré	Core
`webhook_retry`	Toutes les 30min	:00,:30	Retry webhooks échoués	Core
`backup_quotidien`	Quotidien	02:00	Backup base de données	Admin
`optimiser_db`	Hebdo	Sam 03:00	VACUUM, ANALYZE	Admin

Job ID	Fréquence	Heure	Fonction	Module
`rapport_mensuel_budget`	Mensuel	1er, 20:00	Rapport budget mensuel	Finances

10.2 Jobs manquants à implémenter

#	Job proposé	Fréquence	Module	Bénéfice	Phase
JOB-1	**`prediction_courses_weekly`**	Hebdo, Dim 10:00	Cuisine	Pré-remplir liste courses de la semaine basé sur historique	Phase 7
JOB-2	**`sync_jeux_budget`**	Quotidien, 22:00	Jeux → Finances	Synchroniser gains/pertes dans le budget	Phase 7
JOB-3	**`analyse_nutrition_hebdo`**	Hebdo, Dim 20:00	Cuisine	Résumé nutritionnel de la semaine + carences	Phase 7
JOB-4	**`alertes_energie`**	Quotidien, 07:00	Maison	Détection consommation anormale (vs historique)	Phase 7
JOB-5	**`nettoyage_logs`**	Hebdo, Dim 04:00	Admin	Purger logs audit > 90 jours	Phase 7
JOB-6	**`check_garmin_anomalies`**	Quotidien, 08:00	Famille	Alertes si pas d'activité depuis 3 jours	Phase 7
JOB-7	**`resume_jardin_saisonnier`**	Mensuel, 1er, 08:00	Maison	Résumé IA actions jardin du mois + recommandations	Phase 7
JOB-8	**`expiration_documents`**	Quotidien, 09:00	Famille	Rappel documents à renouveler	Phase 7

#	Job proposé	Fréquence	Module	Bénéfice	Phase
				(carte identité, assurance…)	

10.3 Problèmes jobs à corriger

#	Problème	Impact	Fix	Phase
J1	**USER_ID hardcodé "matanne"** dans certains jobs	🔴 Multi-user impossible	Boucler sur tous les users	Phase 1
J2	**Pas d'historique d'exécution**	🔴 Pas de monitoring	Table `job_executions`	Phase 7
J3	**Pas de gestion d'erreur riche** → exceptions loggées mais pas re-tentées	🟡 Jobs silencieusement échoués	Retry avec backoff	Phase 7
J4	**Pas de notification d'échec**	🟡 Pas d'alerte	Notification admin push + email	Phase 7
J5	**Pas de métriques de durée**	🟢 Optimisation impossible	Timer dans `job_executions`	Phase 7

11. Notifications — Plan de consolidation

11.1 Canaux existants (référence)

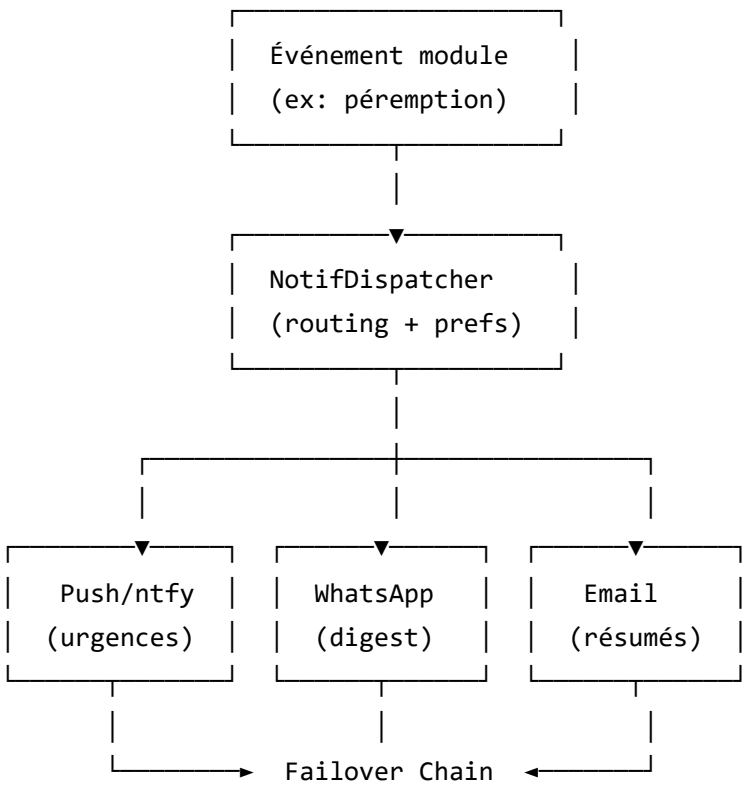
Canal	Module	État	Limitations
ntfy.sh	`src/services/core/notifications/`	✅ Production	Aucune
Web Push (VAPID)	`src/services/core/notifications/`	✅ Production	Fallback mémoire si DB indisponible
Email (Resend)	`src/services/core/notifications/`	✅ Production	Préférences email pas connectées à l'UI

Canal	Module	État	Limitations
WhatsApp (Meta API)	`src/api/routes/webhooks_whatsapp.py`	⚠️ Partiel	Réception OK, envoi incertain, pas de rate limiting

11.2 Gaps notifications à combler

#	Gap	Description	Priorité	Phase
N1	**Pas de failover entre canaux**	Push échoue → pas de fallback WhatsApp/email	HAUTE	Phase 8
N2	**Préférences unifiées manquantes**	Impossible de choisir "WhatsApp urgences, email résumés"	HAUTE	Phase 8
N3	**WhatsApp envoi non confirmé**	Webhook en place mais envoi actif non validé	MOYENNE	Phase 8
N4	**Pas de SMS**	Pas de canal SMS comme fallback ultime	BASSE	Phase 10
N5	**Pas de throttling notifications**	Risque spam si plusieurs modules notifient simultanément	MOYENNE	Phase 8
N6	**Pas de digest**	Pas d'option "résumer les notifications toutes les 2h"	BASSE	Phase 8

11.3 Architecture notification cible



11.4 Mapping événements → canaux cible

Événement	Push	WhatsApp	Email	Fréquence
Péremption J-2	✅	-	-	Quotidien
Rappel courses	✅	✅	-	Quotidien
Résumé hebdo	-	✅	✅	Hebdomadaire
Alerte jeu responsable	✅	✅	-	Immédiat
Rapport budget mensuel	-	-	✅	Mensuel
Anniversaire J-7	✅	✅	-	Ponctuel
Tâche entretien urgente	✅	✅	-	Immédiat
Échec cron job (admin)	✅	✅	✅	Immédiat
Document expirant	✅	-	✅	J-30, J-7
Récolte jardin prête	✅	-	-	Quotidien

12. Mode Admin — Plan d'implémentation

12.1 Fonctionnalités admin existantes (référence)

Endpoint	Action	Déclenchement	État
`POST /api/v1/admin/jobs/{id}/run`	Lancer un cron job manuellement	Manuel (bouton)	✅
`POST /api/v1/admin/notifications/test`	Tester un canal de notification	Manuel	✅
`POST /api/v1/admin/cache/clear`	Vider le cache L1+L3	Manuel	✅
`POST /api/v1/admin/users/{id}/disable`	Désactiver un compte	Manuel	✅
`GET /api/v1/admin/audit-logs`	Consulter les logs d'audit	Lecture	✅
`GET /api/v1/admin/services/health`	État de santé des services	Lecture	✅
`GET /api/v1/admin/db/coherence`	Check cohérence base	Lecture	✅

12.2 Fonctionnalités admin à implémenter

#	Fonctionnalité	Description	Priorité	Phase
ADM-1	**Dashboard admin dédié**	Vue consolidée : derniers logs, jobs statut, health checks, cache stats	HAUTE	Phase 9
ADM-2	**Lancer n'importe quel service manuellement**	Bouton "Run" pour chaque service (pas juste cron jobs)	HAUTE	Phase 9
ADM-3	**Mode dry-run**	Exécuter job/service en mode simulation sans écriture DB	HAUTE	Phase 7
ADM-4	**Historique exécutions jobs**	Table `job_executions` avec start, end, status, error, durée	HAUTE	Phase 7

#	Fonctionnalité	Description	Priorité	Phase
ADM-5	**Toggle feature flags**	Activer/désactiver fonctionnalités depuis admin sans redéployer	MOYENNE	Phase 9
ADM-6	**Voir/modifier configurations**	Éditer paramètres (rate limits, seuils IA, etc.) depuis l'UI	MOYENNE	Phase 9
ADM-7	**Forcer re-sync**	Bouton forcer synchronisation Garmin, Google Calendar, etc.	MOYENNE	Phase 9
ADM-8	**SQL query viewer** (lecture seule)	Exécuter des SELECT depuis l'admin pour debug	BASSE	Phase 10
ADM-9	**Seed data**	Bouton pour injecter données de test (dev uniquement)	BASSE	Phase 10
ADM-10	**Export complet DB**	Télécharger dump complet en JSON/CSV	BASSE	Phase 10

12.3 Principe de visibilité admin

> **Règle** : Les fonctionnalités admin sont **invisibles** pour l'utilisateur normal.
>
> - Les pages `/admin/*` ne sont pas dans la sidebar standard
> - Le header admin n'apparaît que si `role === "admin"`
> - En dev (`ENVIRONMENT=development`), un bouton discret dans le footer permet d'accéder à l'admin
> - Aucun lien direct vers `/admin` dans la navigation mobile

Implémentation frontend à réaliser :

```
// Dans barre-laterale.tsx
{utilisateur?.role === 'admin' && (
  <SidebarGroup label="Administration">
    <SidebarItem href="/admin" icon={Shield} />
  </SidebarGroup>
)}
```

13. Moteur d'automatisation — Plan d'extension

13.1 État actuel (baseline)

Le moteur dans `src/services/utilitaires/automations_engine.py` est **basique** :

Caractéristique	État actuel	Cible
Triggers supportés	1 seul : `stock_bas`	9 triggers
Actions supportées	2 : `ajouter_courses` , `notifier`	11 actions
Historique exécutions	❌ Aucun	✅ Table dédiée
Mode dry-run	❌ Absent	✅ Simulation
Planification par règle	❌ Fréquence fixe globale	✅ Par règle
Rollback	❌ Fire-and-forget	✅ Optionnel
Interface frontend	✅ Page automation existe	✅ Enrichie

13.2 Triggers à ajouter

Trigger	Description	Module source	Phase
`peremption_proche`	Item à J-N de la date de péremption	Cuisine/Inventaire	Phase 7
`budget_depassement`	Budget dépasse le seuil défini	Finances	Phase 7
`meteo_alerte`	Conditions météo défavorables	Maison/Jardin	Phase 7
`anniversaire_proche`	Anniversaire dans N jours	Famille	Phase 7
`tache_en_retard`	Tâche non complétée après la date prévue	Maison/Planning	Phase 7
`garmin_inactivite`	Pas d'activité Garmin depuis N jours	Famille/Santé	Phase 7

Trigger	Description	Module source	Phase
`document_expiration`	Document expire dans N jours	Famille	Phase 7
`recette_sans_photo`	Recette créée sans image depuis N jours	Cuisine	Phase 7

13.3 Actions à ajouter

Action	Description	Module cible	Phase
`generer_liste_courses`	Créer/compléter la liste de courses	Cuisine	Phase 7
`suggerer_recette`	Proposer une recette via push	Cuisine	Phase 7
`creer_tache_maison`	Créer une tâche d'entretien	Maison	Phase 7
`envoyer_whatsapp`	Envoyer un message WhatsApp	Notifications	Phase 8
`envoyer_email`	Envoyer un email formaté	Notifications	Phase 8
`ajouter_au_planning`	Insérer un repas/activité dans le planning	Planning	Phase 7
`mettre_a_jour_budget`	Créer une entrée de dépense	Finances	Phase 7
`generer_rapport_pdf`	Générer et envoyer un rapport	Export	Phase 7
`archiver`	Archiver un élément	Multi	Phase 7

13.4 Architecture cible du moteur

```
AutomationEngine
├── TriggerRegistry        # Enregistre les triggers disponibles
│   ├── StockBasTrigger
│   ├── PeremptionTrigger
│   ├── BudgetTrigger
│   └── ...
├── ActionRegistry         # Enregistre les actions disponibles
│   ├── AjouterCoursesAction
│   ├── NotifierAction
│   ├── SuggererRecetteAction
│   └── ...
├── RuleEngine             # Évalue les règles
│   ├── evaluate_conditions()
│   ├── execute_actions()
│   └── dry_run()
├── ExecutionHistory       # Historique + métriques
│   ├── log_execution()
│   ├── get_history()
│   └── get_stats()
└── Scheduler              # Planification par règle
    ├── schedule_rule()
    └── unschedule_rule()
```

14. Tests — Plan d'amélioration

14.1 Inventaire actuel (baseline)

Catégorie	Fichiers	Tests (est.)	Couverture	Obj. couverture
API routes	~25	~100	70%	85%
Services cuisine	18	~80	80%	90%
Services famille	8	~30	50%	80%
Services maison	5	~20	40%	75%
Services jeux	10	~40	60%	80%

Catégorie	Fichiers	Tests (est.)	Couverture	Obj. couverture
Core (config, db, cache, ai)	~80	~200	85%	90%
Modèles ORM	18	~50	90%	90%
SQL schéma	2	~10	—	—
Benchmarks	5	~15	—	—
Load tests	3	~5	—	—
TOTAL	**~186**	**~550**	**~65%**	**≥80%**

14.2 Couverture par zone critique — cibles

Zone	Couverture actuelle	Objectif	Actions
Auth/JWT	70%	95%	Ajouter edge cases (token expiré, 2FA)
Cron jobs	60%	90%	Gestion erreurs, retry, notifications d'échec
Admin routes	40%	85%	ACL complet, opérations sensibles
WhatsApp/webhook	20%	80%	Mock Meta API, test envoi
Garmin sync	30%	75%	Token refresh, erreurs API
Multi-user	10%	70%	Accès concurrent
WebSocket	30%	70%	Reconnexion, messages perdus
Moteur automations	20%	80%	Tous triggers + actions

14.3 Tests manquants à implémenter

#	Test	Catégorie	Priorité	Phase
T1	**Tests admin ACL** — vérifier user non-admin ne peut pas accéder endpoints admin	API	🔴	Phase 3

#	Test	Catégorie	Priorité	Phase
T2	**Tests cron error handling** — job qui plante → vérifier logging + retry	Services	🔴	Phase 3
T3	**Tests WhatsApp send** — mock Meta API → vérifier formatage message	Intégration	🔴	Phase 3
T4	**Tests multi-user isolation** — 2 users ne voient pas les données de l'autre	Core	🔴	Phase 3
T5	**Tests moteur automations** — triggers + actions + historique	Services	🟡	Phase 3
T6	**Tests WebSocket reconnexion** — déconnexion → reconnexion auto	API	🟡	Phase 3
T7	**Tests inter-modules** — planning → courses → inventaire flow complet	E2E	🟡	Phase 3
T8	**Tests schéma Pydantic ↔ TypeScript** — alignement frontend/backend	Cross	🟡	Phase 3
T9	**Tests performance cron** — durée exécution de chaque job	Benchmark	🟢	Phase 10
T10	**Tests notifications failover** — push échoue → fallback email	Intégration	🟢	Phase 8

14.4 Infrastructure de test à mettre en place

Amélioration	État actuel	Cible	Phase
Coverage report automatique	Manuel (`python manage.py test_coverage`)	CI/CD avec seuil 80%	Phase 3
Mutation testing	Absent	mutmut ou cosmic-ray pour détecter tests faibles	Phase 10
Contract testing	Absent	Pact ou Schemathesis pour valider API contracts	Phase 10

Amélioration	État actuel	Cible	Phase
Visual regression	Absent	Percy ou Chromatic pour screenshots UI	Phase 10
Snapshot testing	Absent	Pour les réponses API JSON complexes	Phase 10

15. Documentation — Plan de mise à jour

15.1 Inventaire docs existantes — état actuel

Document	Contenu	État	Action
`docs/ARCHITECTURE.md`	Architecture technique complète	✅ À jour	Maintenir
`docs/API_REFERENCE.md`	Référence API par endpoint	⚠️ Partiel	Ajouter endpoints récents
`docs/MODULES.md`	Vue d'ensemble des modules	⚠️ Partiel	Mettre à jour post-sprint 14
`docs/SERVICES_REFERENCE.md`	Services et factories	⚠️ Daté	Régénérer depuis le code
`docs/ERD_SCHEMA.md`	Schéma entité-relation	✅ À jour	Maintenir
`docs/PATTERNS.md`	Patterns de code	✅ À jour	Maintenir
`docs/DEPLOYMENT.md`	Guide de déploiement	✅ À jour	Maintenir
`docs/MIGRATION_GUIDE.md`	Guide de migration	✅ À jour	Maintenir
`docs/SQLALCHEMY_SESSION_GUIDE.md`	Gestion sessions DB	✅ À jour	Maintenir

Document	Contenu	État	Action
`docs/REDIS_SETUP.md`	Configuration Redis	✅ À jour	Maintenir
`docs/UI_COMPONENTS.md`	Composants frontend	⚠️ Partiel	Ajouter nouveaux composants

15.2 Guides modules — couverture actuelle

Guide	Couverture	Manque
`docs/guides/cuisine/README.md`	85%	Batch cooking flow, anti-gaspillage IA
`docs/guides/famille/README.md`	75%	Jules IA, gamification, Garmin
`docs/guides/maison/README.md`	70%	Artisans, diagnostics, énergie
`docs/guides/jeux/README.md`	80%	Bankroll management, séries alertes
`docs/guides/planning/README.md`	80%	Sync calendrier, voyages
`docs/guides/dashboard/README.md`	60%	Widgets, cross-module metrics
`docs/guides/outils/README.md`	75%	Automations, assistant vocal
`docs/guides/utilitaires/README.md`	✅ 95%	Complet

15.3 Documentation manquante à créer

#	Document	Priorité	Contenu attendu	Phase
DOC-1	**`docs/ADMIN_RUNBOOK.md`**	🔴 HAUTE	Guide opérationnel : lancer jobs, vider cache, gérer users, diagnostiquer	Phase 4
DOC-2	**`docs/CRON_JOBS.md`**	🔴 HAUTE	Liste des 19+ jobs, horaires, dépendances, troubleshooting	Phase 4
DOC-3	**`docs/NOTIFICATIONS.md`**	🟡 MOYENNE	Canaux, configuration, préférences, failover	Phase 4
DOC-4	**`docs/INTER_MODULES.md`**	🟡 MOYENNE	Cartographie interactions entre modules	Phase 4

#	Document	Priorité	Contenu attendu	Phase
DOC-5	`docs/AI_SERVICES.md`	🟡 MOYENNE	Services IA, prompts, rate limits, cache strategy	Phase 4
DOC-6	`docs/AUTOMATIONS.md`	🟢 BASSE	Moteur d'automatisation, triggers, actions, règles	Phase 4
DOC-7	`docs/TROUBLESHOOTING.md`	🟢 BASSE	Erreurs courantes, solutions, FAQ développeur	Phase 4
DOC-8	`docs/DEVELOPER_SETUP.md`	🟢 BASSE	Installation dev complète pas-à-pas	Phase 4

16. Organisation du code & Résorption de la dette technique

16.1 Points forts de l'architecture (à maintenir)

Aspect	Détail	Score
Modularité	Services isolés, factory pattern, lazy loading	9/10
Patterns cohérents	`@service_factory` , `@gerer_exception_api` , `executer_avec_session`	9/10
Typage fort	Pydantic v2, SQLAlchemy 2.0 `Mapped[]` , TypeScript strict	8/10
Résilience	Circuit breaker, retry, rate limiting, fallback	8/10
Cache multi-niveaux	L1 mémoire → L3 fichier + ETag HTTP + TanStack Query	8/10
Sécurité	JWT + RBAC + RLS + CORS + rate limiting + security headers	9/10

16.2 Dette technique — Plan de résorption

#	Dette	Sévérité	Effort	Phase
DT-1	**30+ `# type: ignore`** — masquent erreurs potentielles. Remplacer par `cast()` ou corriger les types	🟡 MOYENNE	2-3h	Phase 1
DT-2	**50+ stubs `pass`** — fonctions déclarées non implémentées	🟡 MOYENNE	4-6h	Phase 1
DT-3	**8 schémas Pydantic vides** — classes `pass` dans `schemas/utilitaires.py`, `documents.py`	🟡 MOYENNE	1h	Phase 1
DT-4	**Route monolithique `maison.py`** — 35+ endpoints dans un seul fichier	🟢 BASSE	2h	Phase 2
DT-5	**Route monolithique `famille.py`** — 18+ endpoints dans un seul fichier	🟢 BASSE	2h	Phase 2
DT-6	**Client API admin manquant** frontend — pages admin utilisent `clientApi` directement	🟢 BASSE	30min	Phase 2
DT-7	**Types TypeScript incomplets** — `batch-cooking.ts`, `anti-gaspillage.ts` minimaux	🟢 BASSE	1h	Phase 2
DT-8	**Accessibilité frontend** — 20+ violations ARIA dans composants shadcn	🟢 BASSE	2-3h	Phase 2
DT-9	**Pas de lint SQL** — `INIT_COMPLET.sql` non validé par un linter	🟢 BASSE	1h	Phase 10
DT-10	**Pas de pre-commit hooks** — formatage/linting non automatisé au commit	🟢 BASSE	30min	Phase 3

16.3 Organisation des fichiers — Actions

Actuel	Recommandation	Raison	Phase
`src/api/routes/maison.py` (35+ endpoints)	Split en `routes/maison/projets.py`, `routes/maison/entretien.py`, etc.	Maintenabilité	Phase 2

Actuel	Recommandation	Raison	Phase
`src/api/routes/famille.py` (18+ endpoints)	Split en `routes/famille/enfants.py`, `routes/famille/budget.py`, etc.	Maintenabilité	Phase 2
`sql/INIT_COMPLET.sql` (4500 lignes)	Garder tel quel (un seul fichier = source de vérité dev)	Déjà bien structuré	—
Frontend `composants/` flat	Organiser en sous-dossiers par module	Clarté	Phase 10

17. Axes d'amélioration & Innovation

17.1 Innovations techniques

#	Innovation	Description	Impact	Phase
INNO-1	**Mode hors-ligne (PWA)**	Service Worker + IndexedDB pour fonctionner sans connexion (courses, notes, minuteur)	🔴 HAUTE	Phase 10
INNO-2	**Sync temps réel multi-device**	WebSocket pour tous les modules — synchronisation en direct entre téléphone et tablette	🟡 MOYENNE	Phase 10
INNO-3	**Commandes vocales**	L'assistant vocal commande : "Ajoute du lait", "Qu'est-ce qu'on mange ce soir ?"	🟡 MOYENNE	Phase 10
INNO-4	**Widget mobile natif**	PWA widgets ou notifications interactives (bouton "J'ai fait les courses" dans la notification)	🟡 MOYENNE	Phase 10
INNO-5	**Data export/import famille**	Export complet en JSON pour sauvegarde ou partage entre familles	🟢 BASSE	Phase 10

17.2 Innovations fonctionnelles

#	Innovation	Description	Module	Phase
INNO-6	**Planificateur IA semaine complète**	Un bouton : IA génère plannings repas + activités + tâches maison + courses (budget, météo, préférences)	Multi	Phase 10
INNO-7	**Mode "invités"**	Planning repas adapté quand on reçoit (portions, allergies, courses supplémentaires)	Cuisine	Phase 10
INNO-8	**Comparateur de prix**	Intégration API supermarchés pour comparer prix de la liste de courses	Cuisine	Phase 10
INNO-9	**Journal de bord augmenté**	Journal familial avec photos, humeur, météo, activités réalisées — auto-généré	Famille	Phase 10
INNO-10	**Bilan financier annuel**	Rapport IA de fin d'année : tendances, économies, recommandations	Finances	Phase 10
INNO-11	**Carte interactive maison**	Vue 3D/plan avec équipements, tâches en cours, garanties par pièce	Maison	Phase 10
INNO-13	**Suivi écologique**	Score éco : gaspillage alimentaire, consommation énergie, déplacements + badges	Multi	Phase 10
INNO-14	**Calendrier scolaire intégré**	Import auto calendrier scolaire par zone → cron ajuste plannings vacances	Famille	Phase 10
INNO-15	**Mode "urgence"**	Ingrédient manquant au dernier moment → IA propose remplacement + adapte recette	Cuisine	Phase 10

17.3 Améliorations UX

#	Amélioration	Description	Phase
UX-1	**Onboarding interactif**	Tour guidé au premier lancement (composant `tour-onboarding.tsx` existe mais peu utilisé)	Phase 10

#	Amélioration	Description	Phase
UX-2	**Raccourcis clavier**	Cmd+K (recherche globale) existe — ajouter Cmd+N (nouveau), Cmd+P (planning), etc.	Phase 10
UX-3	**Drag & drop planning**	Glisser-déposer les repas dans le planning de la semaine	Phase 10
UX-4	**Thème saisonnier**	Adapter thème couleur en fonction de la saison (spring green, summer gold, etc.)	Phase 10
UX-5	**Dark mode adaptatif**	Bascule auto dark mode selon l'heure (18h-7h)	Phase 10

18. Plan d'action global priorisé par phase

Phase 1 — Stabilisation (Sprint Correctif)

> **Objectif** : Zéro bugs critiques, base solide pour développer.

#	Action	Réf.	Effort	Priorité
1.1	Fix import `ActiviteFamille` dans `recherche.py`	B1	15min	🔴 P0
1.2	Fix cron job `liste_courses` → `articles_courses`	B2	15min	🔴 P0
1.3	Supprimer USER_ID hardcodé dans les cron jobs	B6	1h	🔴 P0
1.4	Compléter les 8 schémas Pydantic `pass`	B4/DT-3	1h	🟡 P1
1.5	Remplacer les 30 `# type: ignore` critiques	B3/DT-1	2h	🟡 P1
1.6	Ajouter guards admin frontend	B11	30min	🟡 P1
1.7	Fix parser IA fallback hardcodé	B5	30min	🟡 P1

Phase 2 — SQL Consolidation + Refactoring léger

> **Objectif** : Schéma SQL propre, cohérent, routes mieux organisées.

#	Action	Réf.	Effort	Priorité
2.1	Standardiser `user_id` → UUID partout	S1/SQL-1	2h	🔴
2.2	Nettoyer tables legacy (doublon courses)	S2/SQL-2	30min	🟡
2.3	Uniformiser CASCADE vs SET NULL	S3/SQL-3	1h	🟡
2.4	Ajouter index manquants (jointures fréquentes)	SQL-4	1h	🟡
2.5	Documenter les vues SQL dans INIT_COMPLET	SQL-5	30min	🟢
2.6	Ajouter contraintes CHECK pour enums	SQL-6	1h	🟢
2.7	Split route `maison.py` en sous-routeurs	DT-4	2h	🟢
2.8	Split route `famille.py` en sous-routeurs	DT-5	2h	🟢
2.9	Client API admin frontend (`admin.ts`)	DT-6	30min	🟢
2.10	Types TypeScript batch-cooking/anti-gaspillage	DT-7	1h	🟢
2.11	Corrections accessibilité ARIA	DT-8/B12	2-3h	🟢

Phase 3 — Tests & Qualité

> **Objectif** : Couverture ≥ 80% sur les zones critiques.

#	Action	Réf.	Effort	Priorité
3.1	Tests admin ACL	T1	2h	🔴
3.2	Tests cron error handling	T2	2h	🔴
3.3	Tests WhatsApp send	T3	1h	🔴
3.4	Tests multi-user isolation	T4	3h	🔴
3.5	Tests moteur automations	T5	2h	🟡
3.6	Tests inter-modules E2E	T7	3h	🟡
3.7	Setup coverage CI/CD avec seuil 80%	—	1h	🟡
3.8	Tests WebSocket reconnexion	T6	1h	🟢
3.9	Setup pre-commit hooks	DT-10	30min	🟢

Phase 4 — Documentation

> **Objectif** : Tout est documenté, à jour, trouvable.

#	Action	Réf.	Effort	Priorité
4.1	Créer `docs/ADMIN_RUNBOOK.md`	DOC-1	2h	🔴
4.2	Créer `docs/CRON_JOBS.md`	DOC-2	1h	🔴
4.3	Mettre à jour les guides modules (cuisine, famille, maison, jeux)	—	4h	🟡
4.4	Créer `docs/NOTIFICATIONS.md`	DOC-3	1h	🟡
4.5	Créer `docs/INTER_MODULES.md`	DOC-4	1h	🟡
4.6	Créer `docs/AI_SERVICES.md`	DOC-5	1h	🟡
4.7	Régénérer `docs/SERVICES_REFERENCE.md`	—	2h	🟢
4.8	Créer `docs/AUTOMATIONS.md`	DOC-6	30min	🟢
4.9	Créer `docs/TROUBLESHOOTING.md`	DOC-7	1h	🟢
4.10	Créer `docs/DEVELOPER_SETUP.md`	DOC-8	1h	🟢

Phase 5 — Interactions inter-modules

> **Objectif** : Les modules communiquent et se renforcent mutuellement.

#	Action	Réf.	Effort	Priorité
5.1	Inventaire péremption → Suggestions recettes	IM-9	3h	🔴
5.2	Jeux pertes/gains → Budget/Finances	IM-8	4h	🔴

#	Action	Réf.	Effort	Priorité
5.3	Routines famille → Planning général	IM-10	4h	🟡
5.4	Jardin récoltes → Cuisine inventaire	IM-12	2h	🟡
5.5	Météo → Activités famille	IM-14	2h	🟡
5.6	Push ↔ WhatsApp failover	IM-15	3h	🟡
5.7	Chat IA contexte multi-module	IM-11	6h	🟢
5.8	Vue budgétaire unifiée dashboard	G-DASH-1	3h	🔴
5.9	Exposer vues SQL dans frontend	G-MAI-3	2h	🟡

Phase 6 — IA & Intelligence

> **Objectif** : L'IA est partout où elle apporte de la valeur.

#	Action	Réf.	Effort	Priorité
6.1	Nutrition → détection carences + suggestions	IA-1	4h	🔴
6.2	Prédiction courses intelligente (avec contexte)	IA-2	4h	🔴
6.3	Assistant multi-contexte	IA-NEW-1	8h	🟡
6.4	Énergie → anomalies IA	IA-7/G-MAI-1	3h	🟡
6.5	Planificateur semaine complète	IA-NEW-3	8h	🟡
6.6	Prédiction péremption (apprentissage patterns)	IA-3	4h	🟢
6.7	Détection de tendances (3-6 mois)	IA-NEW-5	6h	🟢
6.8	Recommandation budget IA	IA-NEW-2	4h	🟢

Phase 7 — Jobs & Automatisations

> **Objectif** : Jobs fiables avec historique, moteur d'automatisation riche.

#	Action	Réf.	Effort	Priorité
7.1	Table `job_executions` + historique automatique	ADM-4/J2	3h	🔴

#	Action	Réf.	Effort	Priorité
7.2	Notification d'échec job → admin (push + email)	J4	1h	🔴
7.3	Ajouter les 8 jobs manquants (JOB-1 à JOB-8)	—	6h	🟡
7.4	Étendre moteur d'automatisation (10 triggers, 9 actions)	—	8h	🟡
7.5	Mode dry-run pour jobs et automations	ADM-3	2h	🟡
7.6	Métriques durée de chaque job	J5	1h	🟢

Phase 8 — Notifications & Communications

> **Objectif** : Notifications fiables, multi-canal, avec préférences utilisateur.

#	Action	Réf.	Effort	Priorité
8.1	Finaliser WhatsApp envoi (confirmer/implémenter)	N3	3h	🔴
8.2	Préférences unifiées par canal et par type	N2	3h	🔴
8.3	Failover push → WhatsApp → email	N1	2h	🟡
8.4	Throttling & digest (regroupement notifications)	N5/N6	3h	🟡
8.5	Mapper événements → canaux (tableau §11.4)	—	2h	🟡
8.6	Tests notifications failover	T10	2h	🟢

Phase 9 — Mode Admin Complet

> **Objectif** : Un admin peut tout contrôler sans toucher au code.

#	Action	Réf.	Effort	Priorité
9.1	Dashboard admin consolidé	ADM-1	4h	🔴
9.2	Lancer tout service manuellement	ADM-2	3h	🔴
9.3	Mode dry-run	ADM-3	2h	🟡
9.4	Feature flags	ADM-5	3h	🟡
9.5	Forcer re-sync externes	ADM-7	1h	🟡

#	Action	Réf.	Effort	Priorité
9.6	Score bien-être interactif	G-DASH-2	2h	🟡
9.7	Page bankroll management	G-JEU-2	3h	🟡
9.8	Tags/catégories notes	G-OUT-1	2h	🟡
9.9	SQL query viewer read-only	ADM-8	3h	🟢
9.10	Seed data dev	ADM-9	2h	🟢

Phase 10 — Innovation & UX

> **Objectif** : Différenciation, expérience utilisateur supérieure.

#	Action	Réf.	Effort	Priorité
10.1	Mode hors-ligne (PWA + IndexedDB)	INNO-1	16h	🟡
10.2	Planificateur IA semaine complète	INNO-6	8h	🟡
10.3	Commandes vocales enrichies	INNO-3	4h	🟡
10.4	Drag & drop planning	UX-3	4h	🟡
10.5	Mode invités	INNO-7	4h	🟢
10.6	Suivi écologique	INNO-13	6h	🟢
10.7	Calendrier scolaire auto	INNO-14	3h	🟢
10.8	Widgets configurables	G-DASH-3	4h	🟢
10.9	Export complet DB	ADM-10	2h	🟢
10.10	Minuteur ↔ recettes	G-OUT-3	2h	🟢
10.11	Mutation testing, contract testing, visual regression	—	8h	🟢

19. Nouveau module — Projet Habitat (Déménagement / Agrandissement / Déco / Jardin)

> **Contexte** : Réflexion sur un 2ème enfant → hésitation entre déménager, faire un agrandissement, ou rester et aménager. Besoin d'un outil pour centraliser la recherche immobilière, la conception de plans, la décoration intérieure et l'aménagement du jardin (2600 m², en pente, tout en longueur).
>
> **Positionnement** : Module **séparé** au même niveau que Cuisine / Famille / Maison (pas un sous-module de Maison). Exception : le tracker de meubles au quotidien reste dans Maison, le module Habitat gère la vision projet.

19.1 Vue d'ensemble du module

Le module **Projet Habitat** est un hub central avec 5 sous-modules :

Sous-module	Description	Écrans
🏠 **Scénarios**	Comparer les 3 options (déménager, agrandir, rester+aménager) avec scoring multicritère	Hub + comparateur
🔍 **Veille Immo**	Scraping direct des sites d'annonces (zone ARA) avec alertes automatiques	Critères, résultats, carte, alertes
📐 **Plans & Modifications IA**	Importer ses plans existants → prompt libre → analyse IA cohérente → génération visuelle	Import, analyse, galerie visuels
🎨 **Déco & Meubles**	Design intérieur par pièce, palette couleurs, génération d'images IA, budget	Pièces, visuels IA, budget
🌳 **Paysagisme**	Aménagement du jardin (2600 m², pente) avec import satellite + canvas zones + visuels IA	Plan jardin, zones, visuels

19.2 Architecture technique proposée

```
# Backend
src/core/models/habitat_projet.py       # Modèles ORM (Scenario, CritereImmo, Plan, Piece, Zone
src/api/routes/habitat.py               # Routes API REST
src/api/schemas/habitat.py              # Schémas Pydantic
src/services/habitat/                   # Services métier
├── scenarios_service.py                # Comparaison scénarios
├── veille_immo_service.py              # Orchestration scraping + scoring + alertes
├── plans_service.py                    # Gestion plans importés + analyse IA
├── deco_service.py                     # Déco intérieure, meubles, visuels IA
├── paysagisme_service.py               # Jardin, plantations, visuels IA
└── habitat_ia_service.py               # IA transversale (estimation, analyse plans)

src/services/integrations/              # Services partagés
├── image_generation.py                 # Service centralisé Hugging Face Inference API
└── scrapers/                           # Scrapers immobiliers
    ├── base.py                         # Classe abstraite ScraperImmo
    ├── leboncoin.py                    # Scraper LeBonCoin
    ├── seloger.py                      # Scraper SeLoger
    ├── pap.py                          # Scraper PAP
    ├── bienici.py                      # Scraper Bien'ici
    └── aggregator.py                   # Agrégateur + déduplication

# Frontend
frontend/src/app/(app)/habitat/
├── page.tsx                            # Hub projet habitat
├── scenarios/page.tsx                  # Comparateur de scénarios
├── veille-immo/page.tsx                # Annonces scrapées + carte + alertes
├── plans/page.tsx                      # Import plans + prompt libre + visuels IA
├── deco/page.tsx                       # Design intérieur par pièce + visuels IA
└── jardin/page.tsx                     # Aménagement paysager + canvas

# SQL
sql/migrations/xxx_habitat_projet.sql   # Tables dédiées
```

19.3 Sous-module : Scénarios — Comparateur de décision

Objectif : Aider à prendre la décision déménager vs agrandir vs rester en pondérant des critères objectifs.

Modèle de données :

```sql
CREATE TABLE habitat_scenarios (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(200) NOT NULL,          -- "Déménager", "Agrandir", "Rester + rénover"
    description TEXT,
    budget_estime DECIMAL(12,2),         -- Budget total estimé
    surface_finale_m2 DECIMAL(8,2),
    nb_chambres INTEGER,
    score_global DECIMAL(5,2),           -- Score calculé 0-100
    avantages JSONB DEFAULT '[]',
    inconvenients JSONB DEFAULT '[]',
    notes TEXT,
    statut VARCHAR(50) DEFAULT 'brouillon',  -- brouillon, actif, archive
    cree_le TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE habitat_criteres (
    id SERIAL PRIMARY KEY,
    scenario_id INTEGER REFERENCES habitat_scenarios(id) ON DELETE CASCADE,
    nom VARCHAR(200) NOT NULL,           -- "Proximité école", "Budget", "Surface jardin"
    poids DECIMAL(3,2) DEFAULT 1.0,      -- Pondération du critère (0.0-1.0)
    note DECIMAL(3,1),                   -- Note 0-10
    commentaire TEXT
);
```

Fonctionnalités :

- Créer N scénarios (déménager 3 pièces, 4 pièces, agrandir extension, agrandir surélévation, rester)
- Définir des critères pondérés (budget, surface, localisation, écoles, transports…)
- Score automatique = Σ(note × poids) / Σ(poids)
- Comparaison visuelle en colonnes (radar chart, barres)
- **IA** : Suggérer les critères pertinents en fonction du contexte (2ème enfant, budget estimé)

19.4 Sous-module : Veille Immobilière — Scraping direct

Objectif : Scraper directement les sites d'annonces immobilières (zone Auvergne-Rhône-Alpes) et alerter quand une bonne affaire apparaît.

Stratégie technique — Scraping direct :

Site	Technique	Anti-ban
LeBonCoin	`httpx` + parsing HTML (`beautifulsoup4`)	Headers rotatifs, délais aléatoires (2-5s), proxy optionnel
SeLoger	API JSON interne (reverse-engineered)	Rate limiting strict (1 req/10s)
PAP	`httpx` + parsing HTML	User-Agent rotatif
Bien'ici	API JSON interne	Rate limiting

Architecture des scrapers :

```python
# src/services/integrations/scrapers/base.py
class ScraperImmo(ABC):
    """Classe abstraite pour tous les scrapers immobiliers."""

    @abstractmethod
    async def scraper_annonces(self, criteres: CriteresImmo) -> list[AnnonceScrapee]: ...

    async def _requete_avec_delai(self, url: str) -> str:
        """Requête HTTP avec délai aléatoire anti-ban (2-5s)."""
        await asyncio.sleep(random.uniform(2, 5))
        async with httpx.AsyncClient(headers=self._headers_rotatifs()) as client:
            response = await client.get(url, timeout=30)
            return response.text

# src/services/integrations/scrapers/aggregator.py
class AggregateurAnnonces:
    """Agrège les résultats de tous les scrapers, déduplique, score."""

    def __init__(self):
        self.scrapers = [LeBonCoinScraper(), SeLogerScraper(), PAPScraper(), BienIciScraper()]

    async def scraper_toutes_sources(self, criteres: CriteresImmo) -> list[AnnonceScrapee]:
        """Lance tous les scrapers en parallèle, déduplique par adresse+surface+prix."""
        ...
```

Cron job :

- **Fréquence** : **1×/jour** (ex: 7h00 du matin) — suffisant pour le besoin
- **Zone** : Auvergne-Rhône-Alpes uniquement (filtrage par département/code postal)

- **Pipeline** : Scraping → Déduplication → Scoring vs critères → Notification si score ≥ seuil
- **Robustesse** : Circuit breaker par scraper (si un site bloque, les autres continuent)

Modèle de données :

```sql
CREATE TABLE habitat_criteres_immo (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(200) NOT NULL DEFAULT 'Recherche principale',
    departements JSONB DEFAULT '[]',     -- ["01","07","15","26","38","42","43","63","69","73",'
    villes JSONB DEFAULT '[]',           -- Villes spécifiques si besoin
    rayon_km INTEGER DEFAULT 10,
    budget_min DECIMAL(12,2),
    budget_max DECIMAL(12,2),
    surface_min_m2 DECIMAL(8,2),
    surface_terrain_min_m2 DECIMAL(10,2),
    nb_pieces_min INTEGER,
    nb_chambres_min INTEGER,
    type_bien VARCHAR(50),               -- maison, appartement, terrain
    criteres_supplementaires JSONB,      -- garage, piscine, plain-pied, etc.
    seuil_alerte DECIMAL(5,2) DEFAULT 0.7, -- Score min pour notification
    actif BOOLEAN DEFAULT TRUE,
    cree_le TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE habitat_annonces (
    id SERIAL PRIMARY KEY,
    critere_id INTEGER REFERENCES habitat_criteres_immo(id),
    source VARCHAR(100) NOT NULL,        -- "leboncoin", "seloger", "pap", "bienici"
    url_source VARCHAR(500) NOT NULL,    -- Lien direct vers l'annonce originale
    titre VARCHAR(500),
    prix DECIMAL(12,2),
    surface_m2 DECIMAL(8,2),
    surface_terrain_m2 DECIMAL(10,2),
    nb_pieces INTEGER,
    ville VARCHAR(200),
    code_postal VARCHAR(10),
    departement VARCHAR(3),
    photos JSONB DEFAULT '[]',
    description_brute TEXT,
    score_pertinence DECIMAL(5,2),       -- Score calculé par rapport aux critères
    statut VARCHAR(50) DEFAULT 'nouveau', -- nouveau, vu, favori, contacte, visite, rejete
    prix_m2_secteur DECIMAL(8,2),        -- Prix au m² de référence DVF
    ecart_prix_pct DECIMAL(5,2),         -- % d'écart par rapport au prix marché
    hash_dedup VARCHAR(64),              -- SHA256(adresse+surface+prix) pour déduplication
    notes TEXT,
    detectee_le TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Fonctionnalités UI :

- Formulaire de critères de recherche (départements ARA, budget, surface, terrain, pièces…)
- Dashboard annonces avec scoring et tri par pertinence
- **Lien direct vers la source** pour chaque annonce (LBC, SeLoger, PAP, Bien'ici)
- Badge "Bonne affaire" si prix < prix_m2_secteur × surface - 10%
- Carte interactive des annonces (Leaflet) — zone ARA
- Pipeline de statut : Nouveau → Vu → Favori → Contacté → Visité → Rejeté / Offre
- Historique des prix DVF du secteur (graphique d'évolution)
- Notification Push quand une annonce "top" est détectée (score ≥ seuil)

19.5 Sous-module : Plans & Modifications IA — Pipeline analyse cohérente

Objectif : Importer ses plans de maison existants, décrire des modifications en langage naturel ("Ajouter une chambre de 12m² côté jardin"), obtenir une **analyse IA complète et cohérente** puis un visuel généré.

Principe clé : Le prompt libre ne génère pas directement une image. Il passe par un **pipeline à 2 étapes** — analyse textuelle par Mistral, puis génération visuelle par Hugging Face.

Pipeline de modification :

```
┌──────────────────────────────────────────────────────────────────┐
│ ÉTAPE 1 — Analyse IA (Mistral)                                   │
│                                                                  │
│ Entrées:                                                         │
│   • Prompt utilisateur: "Ajouter une chambre de 12m² côté jardin" │
│   • Plan importé: pièces existantes, dimensions, positions       │
│   • Contraintes: murs porteurs, PLU, budget, orientation         │
│                                                                  │
│ Sortie: AnalyseModificationPlan (JSON structuré)                 │
│   • faisabilite: bool                                            │
│   • score_confiance: 0.85                                        │
│   • contraintes_identifiees: ["Mur porteur côté est", "PLU: COS"] │
│   • impact_structurel: "Extension légère, fondations nécessaires" │
│   • estimation_cout: {min: 18000, max: 28000}                    │
│   • disposition_proposee: {pièces modifiées avec dimensions}     │
│   • recommandations: ["Privilégier une extension de plain-pied"] │
│   • prompt_image_optimise: "Floor plan, French suburban house..." │
├──────────────────────────────────────────────────────────────────┤
│ ÉTAPE 2 — Génération visuelle (Hugging Face)                     │
│                                                                  │
│ Entrée: prompt_image_optimise (généré par Mistral, pas le brut)  │
│ Sortie: Image architecturale cohérente                           │
│                                                                  │
│ Modèle: SDXL / Stable Diffusion 3 / Flux via HF Inference API    │
└──────────────────────────────────────────────────────────────────┘
```

Modèle Pydantic pour l'analyse :

```python
class AnalyseModificationPlan(BaseModel):
    """Résultat structuré de l'analyse IA d'une modification de plan."""
    faisabilite: bool
    score_confiance: float = Field(ge=0, le=1)
    contraintes_identifiees: list[str]
    impact_structurel: str
    estimation_cout_min: int
    estimation_cout_max: int
    disposition_proposee: dict  # Pièces modifiées avec nouvelles dimensions
    recommandations: list[str]
    prompt_image_optimise: str  # Prompt enrichi pour la génération d'image
    alternatives: list[str] = []  # Suggestions alternatives si faisabilité faible
```

Modèle de données :

```sql
CREATE TABLE habitat_plans (
    id SERIAL PRIMARY KEY,
    scenario_id INTEGER REFERENCES habitat_scenarios(id) ON DELETE SET NULL,
    nom VARCHAR(200) NOT NULL,
    type_plan VARCHAR(50) NOT NULL,       -- 'interieur', 'extension', 'jardin', 'parcelle'
    image_importee_url VARCHAR(500),      -- Plan original importé (photo/scan/PDF)
    donnees_pieces JSONB NOT NULL DEFAULT '{}', -- Pièces extraites/saisies {nom, dims, positio
    contraintes JSONB DEFAULT '{}',       -- Murs porteurs, PLU, orientation, etc.
    surface_totale_m2 DECIMAL(8,2),
    budget_estime DECIMAL(12,2),
    notes TEXT,
    version INTEGER DEFAULT 1,
    cree_le TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    modifie_le TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE habitat_modifications_plan (
    id SERIAL PRIMARY KEY,
    plan_id INTEGER REFERENCES habitat_plans(id) ON DELETE CASCADE,
    prompt_utilisateur TEXT NOT NULL,      -- "Ajouter une chambre de 12m² côté jardin"
    analyse_ia JSONB NOT NULL,            -- AnalyseModificationPlan sérialisé
    image_generee_url VARCHAR(500),       -- URL de l'image générée par HF
    acceptee BOOLEAN,                     -- L'utilisateur a-t-il validé cette modification ?
    cree_le TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE habitat_pieces (
    id SERIAL PRIMARY KEY,
    plan_id INTEGER REFERENCES habitat_plans(id) ON DELETE CASCADE,
    nom VARCHAR(200) NOT NULL,            -- "Chambre bébé", "Bureau", "Salon"
    type_piece VARCHAR(50),               -- chambre, salon, cuisine, sdb, bureau, rangement
    surface_m2 DECIMAL(6,2),
    position_x DECIMAL(8,2),
    position_y DECIMAL(8,2),
    largeur DECIMAL(8,2),
    longueur DECIMAL(8,2),
    hauteur_plafond DECIMAL(4,2) DEFAULT 2.50,
    couleur_mur VARCHAR(7),               -- Hex code
    sol_type VARCHAR(50),                 -- parquet, carrelage, moquette
    meubles JSONB DEFAULT '[]',           -- Liste de meubles placés
    notes TEXT
);
```

Fonctionnalités :

- **Import de plans** : Upload image/scan/PDF des plans existants de la maison
- **Saisie des pièces** : Après import, saisir les pièces et dimensions (l'IA ne peut pas les extraire d'une image)
- **Saisie des contraintes** : Murs porteurs, PLU local, orientation, budget max
- **Prompt libre** : Zone de texte libre → pipeline analyse + visuel
- **Affichage résultat** : Analyse complète (faisabilité, coût, contraintes, recommandations) côte à côte avec le visuel généré
- **Historique** : Toutes les modifications demandées avec leur analyse, pour comparer
- **Versioning** : v1 (état actuel) → v2 (après extension) → comparer
- **Galerie** : Images générées sauvegardées avec leur contexte

19.6 Sous-module : Décoration & Meubles

Objectif : Concevoir la décoration intérieure pièce par pièce, avec génération d'images IA pour se projeter, palette de couleurs, et suivi du budget.

Double présence des meubles :

- **Habitat** : Vision projet — meubles souhaités, inspirations, budget prévisionnel
- **Maison** (module existant) : Tracker quotidien — meubles achetés, entretien, garanties

Quand un meuble est acheté dans Habitat → il est automatiquement créé dans le tracker Maison.

Fonctionnalités :

- **Par pièce** : créer un "projet déco" pour chaque pièce
- **Moodboard** : collecter des inspirations (images uploadées, URLs Pinterest)
- **Palette couleurs** : générer une palette harmonieuse (complémentaire, analogique…) — lib `chroma-js` ou IA
- **Génération visuelle IA** : "Montre-moi ce salon en style scandinave avec du parquet chêne" → image générée via Hugging Face (prompt enrichi par Mistral avec les dimensions/contraintes réelles)
- **Catalogue meubles** : ajouter des meubles souhaités avec prix, dimensions, lien achat, priorité
- **Budget déco** : budget dédié par pièce → suivi achats effectifs vs prévisionnel
- **Sync finances** : Quand un achat est validé avec une date de paiement → auto-sync vers Maison/Finances
- **Checklist emménagement** : liste de ce qu'il faut acheter par priorité (essentiel, confort, cosmétique)

Modèle de données :

```sql
CREATE TABLE habitat_projets_deco (
    id SERIAL PRIMARY KEY,
    piece_id INTEGER REFERENCES habitat_pieces(id) ON DELETE SET NULL,
    nom_piece VARCHAR(200) NOT NULL,
    style VARCHAR(100),                   -- scandinave, industriel, bohème, moderne, classique
    palette_couleurs JSONB DEFAULT '[]',  -- [{nom, hex, role}]
    inspirations JSONB DEFAULT '[]',      -- [{url, image_url, description}]
    budget_prevu DECIMAL(10,2),
    budget_depense DECIMAL(10,2) DEFAULT 0,
    statut VARCHAR(50) DEFAULT 'idee',    -- idee, en_cours, termine
    notes TEXT,
    cree_le TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE habitat_visuels_ia (
    id SERIAL PRIMARY KEY,
    projet_deco_id INTEGER REFERENCES habitat_projets_deco(id) ON DELETE CASCADE,
    plan_id INTEGER REFERENCES habitat_plans(id) ON DELETE CASCADE,
    zone_jardin_id INTEGER REFERENCES habitat_zones_jardin(id) ON DELETE CASCADE,
    prompt_utilisateur TEXT NOT NULL,
    prompt_enrichi TEXT NOT NULL,          -- Prompt optimisé par Mistral
    image_url VARCHAR(500),
    modele_utilise VARCHAR(200),           -- "stabilityai/sdxl-turbo", "black-forest-labs/FLUX
    contexte VARCHAR(50),                 -- "deco", "plan", "jardin"
    cree_le TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE habitat_meubles_souhaites (
    id SERIAL PRIMARY KEY,
    projet_deco_id INTEGER REFERENCES habitat_projets_deco(id) ON DELETE CASCADE,
    nom VARCHAR(200) NOT NULL,
    categorie VARCHAR(100),               -- canapé, table, rangement, luminaire, textile
    dimensions VARCHAR(200),              -- "L180 x P90 x H85 cm"
    prix_estime DECIMAL(10,2),
    prix_reel DECIMAL(10,2),
    url_achat VARCHAR(500),
    image_url VARCHAR(500),
    magasin VARCHAR(200),
    priorite VARCHAR(20) DEFAULT 'moyen', -- essentiel, haut, moyen, bas
    achete BOOLEAN DEFAULT FALSE,
    date_achat DATE,
    date_paiement DATE,                   -- Si renseignée → sync vers Maison/Finances
    synced_maison BOOLEAN DEFAULT FALSE,  -- Créé dans le tracker Maison ?
```

```
    notes TEXT
);
```

19.7 Sous-module : Paysagisme — Aménagement jardin

Objectif : Concevoir l'aménagement du jardin de 2600 m² (terrain en pente, tout en longueur) avec import de vue satellite, canvas zones, visuels IA, et budget.

Fonctionnalités :

- **Import vue satellite** : Upload de la vue satellite de la parcelle comme fond de plan
- **Infos parcelle** : Saisie des données terrain (surface, orientation, pente, type de sol)
- **Plan jardin canvas** : canvas 2D (`react-konva`) par-dessus la vue satellite pour dessiner les zones
- **Zones fonctionnelles** : potager, pelouse, terrasse, aire de jeux, haie, verger, compost, piscine éventuelle…
- **Catalogue plantes** : sélection depuis le catalogue existant (`data/reference/plantes_catalogue.json`) + nouvelles entrées
- **Gestion de la pente** : visualisation du dénivelé, suggestions de terrasses/murets de soutènement
- **Visuels IA** : "Montre-moi cette zone terrasse avec des dalles en pierre et une pergola" → image via Hugging Face
- **Budget paysagisme** : estimation par zone (terrassement, plantations, clôture, arrosage)
- **IA suggestions** : "Propose un aménagement pour un jardin en pente de 2600m² orienté sud avec 2 enfants en bas âge" → plan de zones + estimations
- **Intégration module Jardin existant** : lier les zones du plan paysager aux fiches plantes du module maison/jardin

Modèle de données :

```sql
CREATE TABLE habitat_zones_jardin (
    id SERIAL PRIMARY KEY,
    plan_id INTEGER REFERENCES habitat_plans(id) ON DELETE CASCADE,
    nom VARCHAR(200) NOT NULL,            -- "Potager", "Aire de jeux", "Terrasse haute"
    type_zone VARCHAR(100),               -- potager, pelouse, terrasse, jeux, verger, haie, cor
    surface_m2 DECIMAL(8,2),
    altitude_relative DECIMAL(4,2),       -- Dénivelé par rapport au point bas (en mètres)
    position_x DECIMAL(8,2),
    position_y DECIMAL(8,2),
    largeur DECIMAL(8,2),
    longueur DECIMAL(8,2),
    donnees_canvas JSONB DEFAULT '{}',    -- Forme dessinée sur le canvas (polygone libre)
    plantes JSONB DEFAULT '[]',           -- [{nom, quantite, prix_unitaire}]
    amenagements JSONB DEFAULT '[]',      -- [{type, description, cout}]
    budget_estime DECIMAL(10,2),
    notes TEXT
);
```

19.8 Service génération d'images IA — Hugging Face Inference API (gratuit)

Choix : Hugging Face Inference API (free tier) au lieu de DALL-E 3 / Stability AI payants.

Justification :

- ~1000 requêtes/mois gratuites — largement suffisant pour quelques images/semaine
- Modèles de qualité disponibles : SDXL, Stable Diffusion 3, Flux
- Pas de carte bancaire requise, juste un token HF gratuit
- API simple et légère

Architecture du service centralisé :

```python
# src/services/integrations/image_generation.py
class ServiceGenerationImages:
    """Service centralisé de génération d'images via Hugging Face Inference API."""

    HF_API_URL = "https://api-inference.huggingface.co/models"
    MODELE_DEFAUT = "stabilityai/stable-diffusion-xl-base-1.0"
    MODELES_DISPONIBLES = {
        "sdxl": "stabilityai/stable-diffusion-xl-base-1.0",
        "sdxl-turbo": "stabilityai/sdxl-turbo",
        "flux-dev": "black-forest-labs/FLUX.1-dev",
    }

    async def generer_image(
        self,
        prompt: str,
        modele: str = "sdxl",
        negative_prompt: str = "blurry, low quality, distorted",
    ) -> bytes:
        """Génère une image à partir d'un prompt textuel."""
        url = f"{self.HF_API_URL}/{self.MODELES_DISPONIBLES[modele]}"
        headers = {"Authorization": f"Bearer {settings.HF_API_TOKEN}"}
        payload = {"inputs": prompt, "parameters": {"negative_prompt": negative_prompt}}

        async with httpx.AsyncClient() as client:
            response = await client.post(url, json=payload, headers=headers, timeout=120)
            response.raise_for_status()
            return response.content  # bytes de l'image
```

Configuration :

- Variable d'environnement : `HF_API_TOKEN` dans `.env.local` (token gratuit à créer sur huggingface.co)
- Dépendance Python : `huggingface_hub` (léger) — optionnel, on peut aussi utiliser `httpx` directement
- Rate limiting interne : max 5 images/heure pour ne pas épuiser le quota gratuit
- Fallback : si HF indisponible → message "Service de génération temporairement indisponible"

Usages dans le module Habitat :

- Plans : visuel architectural après analyse de modification
- Déco : projection visuelle d'une pièce dans un style donné
- Jardin : rendu visuel d'une zone aménagée

19.9 Intégrations avec les modules existants

Module existant	Interaction	Direction
Maison/Projets	Les travaux d'agrandissement → projets maison (suivi chantier, artisans)	Habitat → Maison
Maison/Jardin	Les zones jardin du plan → fiches plantes existantes (entretien, arrosage)	Habitat ↔ Jardin
Maison/Meubles	Meuble acheté dans Habitat → auto-créé dans le tracker Maison	Habitat → Maison
Maison/Finances	Budget validé avec date de paiement → agrégé dans les finances maison	Habitat → Finances
Dashboard	Widget "Projet Habitat" : avancement global, prochaines actions, budget restant	Habitat → Dashboard
Famille/Budget	Gros achats meubles/déco → suivi dans le budget famille	Habitat → Famille
IA Chat	Contexte habitat disponible pour le chat IA multi-module	Habitat → Chat IA
Notifications	Alertes annonces immobilières top → Push/WhatsApp avec lien source	Habitat → Notifications

19.10 IA — Opportunités spécifiques

#	Opportunité IA	Description	Phase
HAB-IA-1	**Analyse modification plan**	Prompt libre → analyse faisabilité, contraintes, coût, disposition — pipeline Mistral structuré	H6
HAB-IA-2	**Génération visuelle plan**	Prompt optimisé par Mistral → image architecturale via Hugging Face	H6
HAB-IA-3	**Estimation budget travaux**	Plan + type travaux → estimation réaliste (coût/m² régional ARA)	H10
HAB-IA-4	**Suggestions déco + visuel**	Style + pièce + contraintes → palette + suggestions + image de projection	H7

#	Opportunité IA	Description	Phase
HAB-IA-5	**Analyse annonce immobilière**	Résumé auto annonce scrapée + scoring + comparaison au marché DVF	H10
HAB-IA-6	**Plan jardin intelligent**	Contraintes (pente, orientation, budget, enfants) → proposition zones + estimations	H8
HAB-IA-7	**Architecte virtuel**	"J'ai 30m² de plus, comment les répartir ?" → analyse + propositions d'agencement	H10
HAB-IA-8	**Visuel paysager**	Zone jardin + aménagements → image de rendu du jardin aménagé	H8

19.11 Budget — Stratégie de gestion

Budget dédié au module Habitat, avec **auto-sync vers Maison/Finances** :

- Chaque sous-module a son propre budget (scénarios, déco par pièce, jardin par zone, travaux)
- Budget global Habitat = somme des sous-budgets
- **Quand un achat est validé avec une date de paiement** → écriture automatique dans Maison/Finances avec :
  - Catégorie : "Habitat / {sous-module}" (ex: "Habitat / Déco salon")
  - Montant, date de paiement, description
  - Lien retour vers l'item Habitat d'origine
- Vue consolidée : budget prévu vs dépensé vs reste à engager

19.12 Plan d'implémentation Habitat

Phase	Contenu	Effort estimé	Priorité
H1	Tables SQL + modèles ORM + schémas Pydantic + routes API CRUD	6h	🔴
H2	Hub frontend + page Scénarios (comparateur)	4h	🔴
H3	Scrapers immobiliers (LBC, SeLoger, PAP, Bien'ici) + agrégateur + cron 1×/jour	10h	🔴
H4	Veille Immo frontend : liste annonces, carte Leaflet, alertes, lien source	6h	🔴

Phase	Contenu	Effort estimé	Priorité
H5	Service Hugging Face Inference API (génération images centralisé)	3h	🔴
H6	Plans : import + saisie pièces/contraintes + pipeline analyse IA Mistral + visuel HF	10h	🟡
H7	Déco & Meubles : projets par pièce, visuels IA, moodboard, budget, sync Maison	8h	🟡
H8	Paysagisme : import satellite + canvas Konva + zones + visuels IA + catalogue plantes	8h	🟡
H9	Budget dédié + auto-sync Maison/Finances sur date de paiement	4h	🟡
H10	IA avancée : estimation travaux régionale, architecte virtuel, analyse annonces	6h	🟢
H11	Intégrations inter-modules (projets, jardin, finances, dashboard, notifications)	4h	🟢
H12	Historique DVF (data.gouv.fr) + graphiques évolution prix secteur	4h	🟢

19.13 Dépendances npm/pip à ajouter

Package	Usage	Côté
`react-konva + konva`	Canvas 2D (plan jardin, zones)	Frontend
`chroma-js`	Manipulation de couleurs (palettes déco)	Frontend
`react-leaflet + leaflet`	Carte interactive annonces (zone ARA)	Frontend
`httpx`	Requêtes HTTP async scrapers + Hugging Face API	Backend
`beautifulsoup4`	Parsing HTML pages annonces (LBC, PAP)	Backend
`huggingface_hub`	Client HF Inference API (optionnel, `httpx` suffit)	Backend

Annexe A — Tableau de synthèse des fichiers clés

Backend

Fichier	Rôle	État	Action
`src/api/main.py`	Point d'entrée FastAPI	✅	Maintenir
`src/api/routes/` (36 fichiers)	Routeurs REST	✅	+ habitat.py
`src/api/schemas/` (23 fichiers)	Schémas Pydantic	⚠️ 8 stubs	Compléter (Phase 1)
`src/api/dependencies.py`	Auth JWT + RBAC	✅	Maintenir
`src/core/models/` (30 fichiers)	Modèles SQLAlchemy	✅	+ habitat_projet.py
`src/core/ai/`	Client Mistral + cache + resilience	✅	Maintenir
`src/core/caching/`	Cache L1/L3	✅	Maintenir
`src/core/db/`	Engine + sessions + migrations	✅	Maintenir
`src/services/core/cron/jobs.py`	19 cron jobs	⚠️ Bugs B2, B6	Fix (Phase 1)
`src/services/core/registry.py`	Service factory registry	✅	Maintenir

Frontend

Fichier	Rôle	État	Action
`frontend/src/app/(app)/`	57 pages applicatives	✅	+ pages Habitat
`frontend/src/bibliotheque/api/`	23 clients API	⚠️ Manque admin.ts	+ admin.ts + habitat.ts
`frontend/src/types/`	13 fichiers types TS	⚠️ batch-cooking minimal	Compléter (Phase 2)
`frontend/src/crochets/`	12 hooks React	✅	Maintenir

Fichier	Rôle	État	Action
`frontend/src/magasins/`	4 stores Zustand	✅	Maintenir
`frontend/src/composants/ui/`	29 shadcn/ui	⚠️ 20 violations ARIA	Fix (Phase 2)
`frontend/src/composants/disposition/`	17 composants layout	✅	Maintenir

Infrastructure

Fichier	Rôle	État	Action
`sql/INIT_COMPLET.sql`	Schéma DB complet	⚠️ Incohérences S1-S5	Consolider (Phase 2)
`tests/` (186 fichiers)	Suite de tests	⚠️ Couverture 65%	→ 80% (Phase 3)
`docs/` (25 fichiers)	Documentation	⚠️ Guides datés	Mettre à jour (Phase 4)
`pyproject.toml`	Config Python/tools	✅	Maintenir
`pytest.ini`	Config tests	✅	Maintenir
`.github/workflows/`	CI/CD	✅	Ajouter seuil coverage

Annexe B — Checklist review avant implémentation

Avant de commencer chaque phase, vérifier :

- ☐ Les tests existants passent (`pytest --tb=short`)
- ☐ Le frontend build (`cd frontend && npx next build`)
- ☐ Pas de régressions sur les endpoints critiques
- ☐ La doc est mise à jour avant de merge
- ☐ Le `INIT_COMPLET.sql` est cohérent avec les modèles SQLAlchemy
- ☐ Les nouveaux endpoints ont des schémas Pydantic + types TypeScript

- [ ] Les nouveaux services utilisent `@service_factory`
- [ ] Les nouveaux cron jobs ont des tests unitaires
- [ ] Les interactions inter-modules utilisent le bus d'événements (pas d'appels directs)
- [ ] Le mode admin est invisible pour l'utilisateur standard

Planning d'implémentation généré le 29 mars 2026 — Basé sur l'audit exhaustif ANALYSE_COMPLETE.md
À utiliser comme référence pour chaque sprint d'implémentation